Murilo Della Coletta, Daniel Henrique Charlo e Otávio Saturnino

SGMP

Limeira-SP

2026

Sumário

1 Introdução 3
2 Levantamento de Requisitos 3
2.1 Requisitos Funcionais 3
2.2 Requisitos Não Funcionais 4
2.3 MoSCoW 4
3 Escopo Técnico 5
4 Arquitetura da Aplicação 5
5 Comunicação RESTful 5
6 Autenticação e Autorização 5
7 Validação de Dados 5
8 Workflow de Chamados 5
9 Notificações de Progresso 6
10 Boas Práticas Adotadas 6
11 Versionamento e Entrega 6
12 Metodologia Ágleis (KANBAN) 6
13 Diagrama de Classes 6
14 Diagrama de Sequência 7

1 Introdução

Este documento descreve de forma detalhada a documentação técnica da API PredialFix. O foco está exclusivamente na lógica de funcionamento, arquitetura, fluxo de dados, segurança e organização do back-end, sem abordar a implementação de banco de dados, que será definida em etapa posterior do projeto.

2 Levantamento de Requisitos

O levantamento de requisitos consiste na etapa de identificação, análise e documentação das necessidades que o sistema deve atender. Essa fase é fundamental para garantir que o sistema desenvolvido esteja alinhado com os objetivos e necessidades dos usuários e da organização.

Durante o processo de levantamento foram definidos requisitos funcionais e requisitos não funcionais. Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer, ou seja, as ações e serviços que o sistema deve executar, como cadastro de usuários, abertura de ordens de serviço e registro de ativos.

Já os requisitos não funcionais definem as características de qualidade do sistema, como desempenho, segurança, disponibilidade e usabilidade. Esses requisitos estabelecem restrições e padrões que devem ser seguidos durante o desenvolvimento do sistema.

Para auxiliar na priorização das funcionalidades foi aplicada a técnica MoSCoW, que classifica os requisitos em quatro categorias: Must Have, Should Have, Could Have e Won't Have**,** permitindo identificar quais funcionalidades são essenciais para o funcionamento do sistema e quais podem ser implementadas em versões futuras.

Essa etapa é essencial para garantir que o desenvolvimento ocorra de forma estruturada, reduzindo riscos e aumentando a qualidade do produto final.

2.1 Requisitos Funcionais

REQUISITOS FUNCIONAIS
• RF01 – O sistema deve permitir cadastro de usuários. • RF02 – O sistema deve permitir edição e exclusão de usuários. • RF03 – O sistema deve permitir diferentes perfis de acesso (Administrador, Técnico, Solicitante). • RF04 – O sistema deve permitir autenticação por login e senha. • RF05 – O sistema deve permitir recuperação de senha. • RF06 – O sistema deve registrar logs de acesso. • RF07 – O sistema deve permitir cadastro de ativos (equipamentos e estruturas). • RF08 – O sistema deve permitir editar e excluir ativos. • RF09 – O sistema deve registrar localização do ativo. • RF10 – O sistema deve registrar número de patrimônio. • RF11 – O sistema deve manter histórico de manutenção por ativo. • RF12 – O sistema deve permitir abertura de ordem de serviço. • RF13 – O sistema deve permitir classificar a OS (preventiva, corretiva, emergencial). • RF14 – O sistema deve permitir anexar fotos e documentos à OS. • RF15 – O sistema deve permitir definir prioridade da OS. • RF16 – O sistema deve permitir alterar o status da OS (Aberta, Em andamento, Concluída, Cancelada). • RF17 – O sistema deve registrar data, hora e responsável por cada alteração de

status.

- RF18 – O sistema deve gerar número único para cada OS.
- RF19 – O sistema deve manter histórico completo da OS.
- RF20 – O sistema deve permitir encerramento da OS com descrição do serviço executado.
- RF21 – O sistema deve permitir cadastro de planos de manutenção preventiva.
- RF22 – O sistema deve permitir definir periodicidade das manutenções.
- RF23 – O sistema deve gerar automaticamente ordens de serviço com base no cronograma.
- RF24 – O sistema deve emitir alertas de manutenção programada.
- RF25 – O sistema deve permitir registrar custos de peças.
- RF26 – O sistema deve permitir registrar custos de mão de obra.
- RF27 – O sistema deve vincular custos a uma ordem de serviço.

2.2 Requisitos Não Funcionais

REQUISITOS NÃO FUNCIONAIS

- RNF01 – O sistema deve possuir controle de acesso baseado em perfil.
- RNF02 – O sistema deve armazenar senhas de forma criptografada.
- RNF03 – O sistema deve suportar no mínimo 100 usuários simultâneos.
- RNF04 – O tempo de resposta das operações deve ser inferior a 3 segundos.
- RNF05 – O sistema deve processar geração de relatórios em até 5 segundos.
- RNF06 – O sistema deve possuir disponibilidade mínima de 99%.
- RNF07 – O sistema deve realizar backup automático diário.
- RNF08 – O sistema deve permitir restauração de dados em caso de falha.
- RNF09 – O sistema deve possuir interface responsiva.
- RNF10 – O sistema deve ter interface intuitiva e de fácil navegação.
- RNF11 – O sistema deve possuir arquitetura escalável.
- RNF12 – O sistema deve permitir atualização sem perda de dados.

2.3 MoSCoW

	Must Have	Should Have	Could Have	Won't Have
Requisitos Funcionais	RF01 RF03 RF04 RF07 RF09 RF10 RF12 RF13 RF15 RF16 RF18 RF20	RF02 RF08 RF11 RF17 RF19 RF21 RF22	RF05 RF06 RF14 RF23 RF24 RF25 RF26 RF27	
Requisitos Não Funcionais	RNF01 RNF02 RNF04	RNF03 RNF05 RNF09	RNF08 RNF11	

	RNF07	RNF10		

3 Escopo Técnico

- Receber e processar requisições HTTP
- Validar dados de entrada
- Controlar autenticação e autorização
- Gerenciar fluxo de status dos chamados
- Garantir segurança e integridade das operações

4 Arquitetura da Aplicação

A aplicação é estruturada em camadas bem definidas, mesmo utilizando PHP puro, seguindo boas práticas de separação de responsabilidades:

- Entry Point: arquivo index.php responsável por receber todas as requisições.
- Router: responsável por identificar a rota e método HTTP.
- Controllers: camada responsável por coordenar ações.
- Services: implementação das regras de negócio.
- Middlewares: validações globais como autenticação.
- Utils: funções auxiliares reutilizáveis.

5 Comunicação RESTful

A API segue os princípios REST:

- Uso correto dos métodos HTTP (GET, POST, PUT)
- Recursos identificados por URLs
- Comunicação

6 Autenticação e Autorização

A autenticação é baseada em token de acesso enviado no header Authorization (Bearer Token). Cada requisição protegida passa obrigatoriamente por um middleware de autenticação.

A autorização é feita com base no nível do usuário:

- Usuário comum: abertura e visualização de chamados
- Responsável técnico: atualização de status e acompanhamento

7 Validação de Dados

Toda entrada de dados é validada manualmente antes do processamento, garantindo:

- Campos obrigatórios
- Tipos de dados corretos
- Limites de tamanho
- Sanitização para evitar dados maliciosos
- Requisições inválidas retornam erro.

8 Workflow de Chamados

O fluxo de atendimento segue obrigatoriamente a sequência:

Aberto → Em Análise → Em Execução → Concluído

Não é permitido pular etapas. Tentativas de transição inválida retornam erro.

9 Notificações de Progresso

As notificações são simuladas e retornadas como mensagens informativas nas respostas da API, refletindo o estágio atual do chamado, como:

- Chamado em análise
- Técnico a caminho
- Serviço finalizado

10 Boas Práticas Adotadas

- Código organizado e modular
- Padronização de respostas
- Separação clara de responsabilidades
- Facilidade de manutenção e expansão
- Preparado para integração futura com banco de dados

11 Versionamento e Entrega

- O projeto deve ser versionado em repositório GitHub.
- Commits descritivos e organização clara de pastas.

12 Metodologia Ágleis (KANBAN)

Para o desenvolvimento do sistema foi adotada a metodologia ágil Kanban, que tem como objetivo organizar e controlar o fluxo de trabalho de forma visual, permitindo maior eficiência na execução das atividades e acompanhamento contínuo das tarefas.

O Kanban é baseado na utilização de um quadro visual, no qual as atividades são distribuídas em colunas que representam as etapas do processo de desenvolvimento, como por exemplo: A Fazer, Em Andamento e Concluído. Cada tarefa é representada por um cartão que se movimenta entre essas colunas conforme o progresso do trabalho.

A utilização dessa metodologia possibilita uma melhor organização das atividades, maior transparência do andamento do projeto e facilita a identificação de possíveis gargalos no processo de desenvolvimento. Além disso, o Kanban permite que a equipe acompanhe de forma simples o status de cada tarefa, garantindo maior controle e produtividade.

Outro benefício do uso do Kanban é a flexibilidade na gestão das demandas, permitindo priorizar atividades de acordo com as necessidades do projeto, bem como realizar ajustes no planejamento durante o desenvolvimento do sistema.

Trello

13 Diagrama de Classes

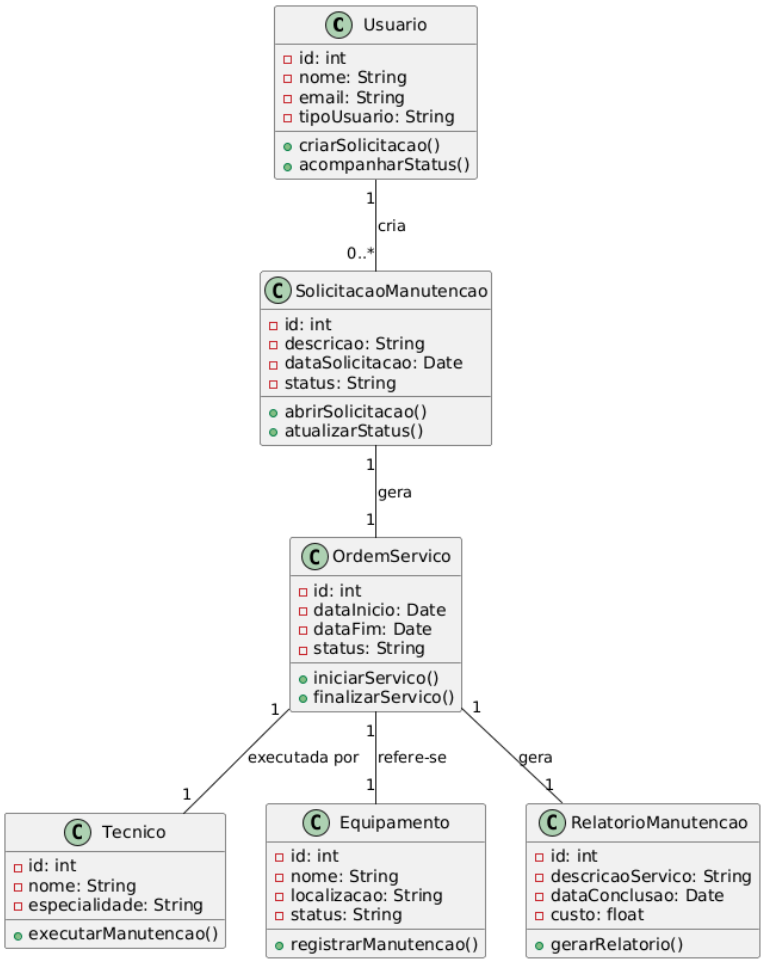
O Diagrama de Classes é um dos principais diagramas da UML (Unified Modeling Language) e tem como objetivo representar a estrutura estática do sistema, descrevendo as classes existentes, seus atributos, métodos e os relacionamentos entre elas.

No contexto do sistema de Gestão de Manutenção Predial, o diagrama de classes foi utilizado para modelar os principais elementos que compõem o sistema, como usuários, ativos, ordens de

serviço e técnicos responsáveis pelas manutenções. Cada classe representa uma entidade do sistema e contém atributos que descrevem suas características, além de métodos que definem suas funcionalidades.

Além disso, o diagrama também apresenta os relacionamentos entre as classes, permitindo visualizar como os diferentes componentes do sistema se conectam e interagem entre si. Por exemplo, um usuário pode criar solicitações de manutenção, uma ordem de serviço está associada a um ativo específico e um técnico pode ser responsável pela execução da manutenção.

A utilização do diagrama de classes contribui para uma melhor organização da arquitetura do sistema, facilitando o desenvolvimento do software e a compreensão da estrutura do sistema por parte da equipe de desenvolvimento.

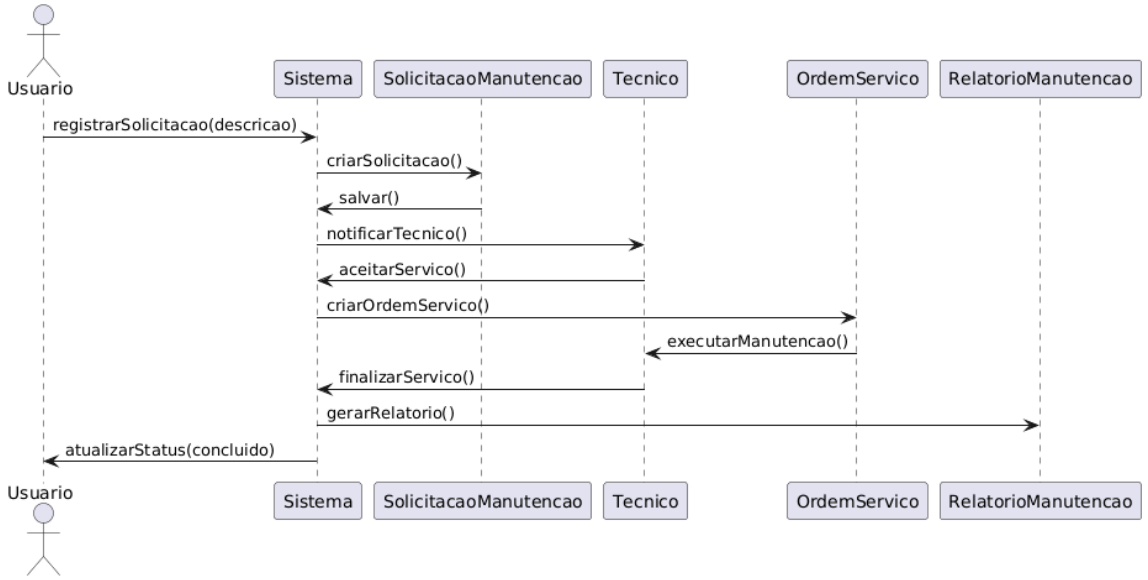


14 Diagrama de Sequência

O Diagrama de Sequência é um diagrama comportamental da UML utilizado para representar a interação entre os elementos do sistema ao longo do tempo. Esse tipo de diagrama demonstra como os objetos ou atores do sistema se comunicam por meio de mensagens para realizar determinada funcionalidade.

No sistema de Gestão de Manutenção Predial, o diagrama de sequência foi utilizado para representar o fluxo de interação entre o usuário, o sistema e os demais componentes durante a execução de processos importantes, como a abertura e o gerenciamento de uma ordem de serviço.

Nesse diagrama é possível observar a sequência de mensagens trocadas entre os participantes do sistema, desde o momento em que o usuário registra uma solicitação de manutenção até a criação

da ordem de serviço, a execução do trabalho pelo técnico e o encerramento do serviço com a geração do relatório final.

A utilização do diagrama de sequência permite compreender de forma clara o fluxo de funcionamento do sistema, facilitando a análise dos processos, a identificação de possíveis melhorias e o alinhamento entre os requisitos do sistema e sua implementação.